EV BUSINESS PLATFORM

CLIENT & DEVELOPER BLUEPRINT

Product Requirements • UX Architecture • Database • Backend • Workflows • Security • QA

Document version: 0.1 — Consolidated Planning Blueprint
Status: Draft for client review and developer estimation
Date: 1 November 2026

Planning document only. This is not build authorization, final legal specification, or proof that any system has been implemented.

Document Control & How to Use This Blueprint

Purpose: one shared reference for the client, product owner, UI/UX designer, developers, QA and deployment teams. It consolidates the planning work completed so far.

Field	Value
Status	Draft v0.1 — review and estimate; not yet client-approved
Project	EV customer, service-centre and spare-parts ecosystem
Interfaces	Android app, customer website, service-centre portal, admin portal
Implementation	Planning only: no code, database, infrastructure or deployment authorized
Source basis	Prior planning discussion and visual interpretation of O7A.pdf. Its text was not reliably extractable; screen labels below are interpreted reference content.

Status labels

Label	Meaning
Proposed	Recommended approach; reviewable.
Assumption	Temporary working assumption.
Pending	Needs client/business confirmation.
Confirmed from discussion	Explicitly stated or accepted in planning.
Not in scope yet	Do not implement without separate approval.

Developers must not silently convert assumptions into fixed business rules. Record scope, cost, timeline, integration and data-model changes through change control.

1. Executive Summary

The proposed platform is a connected EV business ecosystem serving vehicle owners, service centres, spare-parts operations and central administrators. It brings vehicle registration and information, service booking, service progress, support/escalation, educational videos and spare-parts purchasing into one coordinated system.

• Customer-facing Android app and responsive website.

• Service-centre portal for appointments, work orders, technicians, stock and local sales.

• Central admin portal for organizations, users, vehicles, centres, services, parts, orders, payments, media, support, reports and configuration.

• One shared backend and relational database with role-based access and business/centre-level data isolation.

• Online and offline spare-parts sales channels, subject to final business rules.

• Modular Monolith recommended for the first release; avoid premature microservices.

Product principle: keep customer tasks simple while operational workflows, inventory movements, permissions and audit trails remain controlled and traceable.

2. Product Vision, Goals & Scope

Vision

Create a reliable digital platform connecting EV customers with vehicle information, service support, authorized service centres and compatible spare parts, while giving operators a controlled way to manage service operations and commerce.

Business goals

• Simplify customer and EV registration.

• Enable service-centre discovery and service booking.

• Track appointments, work orders and service history.

• Provide video guidance and demo content.

• Enable compatible spare-parts discovery and purchase.

• Support offline/service-centre sales with stock traceability.

• Give administrators operational visibility.

Proposed scope

Area	Initial scope direction
Customer	Account, vehicle garage, vehicle information, booking, tracking, videos, support and parts shopping.
Service centre	Appointments, customer/vehicle records, work orders, technicians, parts usage, inventory and offline sales.
Admin	Organizations, users/roles, centres, services, vehicles, parts, inventory, orders, payments, media, support, reports and audit.
Platform	Shared APIs, PostgreSQL, secure authentication, object storage, notifications, logs and backups.
Later / optional	Telematics/live monitoring, OTA firmware, advanced AI video assistant, complex marketplace.

Out of scope until approved

• Live battery telemetry or GPS tracking without confirmed device/API integration.

• Remote firmware/OTA updates or vehicle ECU control.

• Automatic AI diagnosis or guaranteed repair recommendations.

• Multi-vendor settlement/commission engine unless required by the business model.

• Any payment, WhatsApp or external integration provider not yet selected.

3. Reference Screens & Functional Modules

These labels were previously interpreted from the visual reference. They are a starting point, not a verified textual transcription of the PDF.

Reference area	Interpreted functions
Main Dashboard	Customer • Spare Parts • Service Center
Customer	Scanner; Registration; Video Assistants; Demo Video; Call + Visit; Transfer to Higher Department; Vehicle Related Information; Monitoring Options.
Service Center	Video Assistants; Demo Video; Spare Parts; Software Section Service; Book Appointment.
Service Spare Parts	Online Purchase Options; Offline Purchase Options.

Customer module

• Account registration and profile management.

• Add/manage EVs; view model, variant, identifiers, warranty and service history where available.

• Book service and track booking/work-order progress.

• Browse videos and demo material.

• Contact a centre or submit support/escalation requests.

• Browse compatible parts and view order history.

• Scanner purpose and scan target are Pending.

• Monitoring Options meaning is Pending; no live telemetry is assumed.

Service-centre module

• Dashboard for appointments, work orders, technicians, low stock and recent activity.

• Customer and vehicle records relevant to the centre.

• Appointment review, confirmation, cancellation and rescheduling.

• Work orders, tasks, technician assignment, parts usage, inspection and service report.

• Stock-in, stock-out, transfers and adjustments with movement history.

• Online fulfilment and offline counter sales, if enabled.

• Software-service scope remains Pending.

Admin module

• Organization/brand management; customers, users, staff, roles and permissions.

• Vehicle brands/models/variants; service-centre onboarding and service catalogue.

• Appointments, work orders and technician oversight.

• Parts catalogue, compatibility, pricing, inventory and suppliers.

• Orders, offline sales, payments, invoices and refunds.

• Video library, support, escalations, notifications, analytics, settings and audit logs.

4. User Roles & Access Model

Role	Typical responsibilities
Super Admin	System-wide configuration, organizations, security and platform oversight.
Business Admin	Own organization, brands, users, centres, catalogue, reports and settings.
Service Centre Owner	Own centre operations, staff, appointments, work orders, stock and sales.
Service Manager	Daily service operations, bookings, work-order oversight and quality checks.
Technician	Assigned work orders, inspections, tasks, parts usage and service notes.
Inventory / Parts Manager	Parts catalogue, stock, movements, transfers and reconciliation.
Customer	Own profile, vehicles, bookings, orders, payments, support and personal records.

Authorization must be enforced server-side for every request and resource. Hiding a button is not security. Organization scope and service-centre scope must be checked independently.

5. Application & Navigation Architecture

Customer Android app

• Proposed bottom navigation: Home, Garage, Service, Shop, Profile.

• Home: Book Service, Spare Parts, Video Assistant, Nearby Centres and current vehicle/service information.

• Garage: vehicle list, add vehicle, details, documents, warranty and history.

• Service: centre discovery, service selection, booking and tracking.

• Shop: parts categories, search, details, compatibility, cart, checkout and orders.

• Profile: account, notifications, support, settings and privacy.

Customer website

• Public: Home, About, Services, Spare Parts, Product Details, Service Centres, Centre Details, Videos/Learning, Contact, Login/Register.

• Authenticated: Dashboard, Vehicles, Vehicle Details, Book Service, Appointments, Orders, Tracking, Profile and Support.

• Responsive layout; preserve key tasks on mobile browsers.

Service-centre portal

• Overview; appointments; customers; vehicles; work orders; technicians; parts; inventory; offline sales; videos; reports; settings.

Admin portal

• Overview; organizations; brands; customers; users/staff; vehicles; centres; service catalogue; appointments; work orders; technicians; parts; inventory; online orders; offline sales; payments; invoices; videos; support/escalations; reports; notifications; roles/permissions; settings; audit.

6. UI/UX Design System

Proposed visual direction: modern EV/service interface with blue and dark-navy accents, clear cards, practical icons, readable tables and restrained status colours.

Token	Proposed value	Use
Primary blue	#24558D	Primary actions, links, selected navigation.
Dark navy	#15243B	Headers, navigation and strong text.

Token	Proposed value	Use
Background	#F5F7FA	Page background.
Surface	#FFFFFF	Cards, forms and dialogs.
Success	#16A36A	Completed/verified states.
Warning	#D97706	Attention and pending states.

These are approximate proposals, not verified colour extraction from the PDF.

Accessibility & responsive behaviour

• Target WCAG 2.2 AA where practical.

• Keyboard navigation and visible focus on web portals.

• Screen-reader labels and meaningful image alt text.

• Mobile-friendly touch targets.

• Mobile single column; tablet two columns; desktop sidebar and data tables.

• Loading, empty, error, offline, success, unauthorized, permission-denied and not-found states.

7. Recommended Technical Architecture

Layer	Recommended direction	Notes
Android	React Native + TypeScript	Validate device requirements before final selection.
Website & portals	Next.js + TypeScript	Customer, service-centre and admin portals.
Backend	NestJS + Node.js	Modular Monolith, REST API and centralized authorization.
Database	PostgreSQL	Relational data, constraints, transactions and reporting.
ORM / migrations	Prisma (proposed)	Typed access and versioned migrations.
Authentication	Managed secure auth provider or vetted implementation	Provider selection Pending; never store plaintext passwords.
Media	Private object storage + CDN	Images, documents and videos; signed access URLs.
Background jobs	Queue worker; Redis if justified	Notifications, media processing, reports and retries.
API docs	OpenAPI / Swagger	Contract visibility for clients and integrations.
Hosting	Cloud platform with managed database/storage	Provider, region, cost and compliance Pending.

Logical architecture

Android / Customer Website / Service-Centre Portal / Admin Portal
↓ HTTPS REST API
Backend modules: Identity & Access • Organizations • Customers • Vehicles • Service Network • Appointments • Work Orders • Inventory • Parts & Commerce • Payments • Media • Support • Notifications • Reports
↓
PostgreSQL + private object storage + background worker + external integrations.

Architecture decisions

• Start with a Modular Monolith; keep clear internal module boundaries.

• One shared backend/API for all interfaces.

• Use organization_id and centre-level scope for business-owned records.

• Use transactions for stock changes, order creation and payment state transitions.

• Use /api/v1, pagination, filtering and consistent error responses.

• Keep secrets server-side; never embed private keys in client apps.

8. Database Architecture — Logical Data Domains

PostgreSQL is recommended because the platform contains connected entities, financial records, inventory movements, appointments and auditable workflows. This is a logical design, not a final physical schema.

Domain	Representative entities
Organization	organizations, brands, organization_brands, service_centers, center_brands
Identity	users, profiles, roles, permissions, user_roles, organization_members, center_staff
Customer	customer_profiles, addresses, preferences
Vehicles	vehicle_categories, vehicle_brands, vehicle_models, vehicle_variants, vehicles, identifiers, documents, warranties, service_history
Service Network	center_addresses, center_contacts, center_services, business_hours, service_catalog
Service Operations	service_bookings, booking_status_history, work_orders, tasks, work_order_parts, technician_assignments, service_reports
Inventory	parts, categories, variants, compatibility, pricing, suppliers, inventory_locations, inventory, stock_movements, stock_transfers, purchase_orders
Commerce	carts, cart_items, orders, order_items, order_status_history, sales, sales_items, delivery_addresses, shipments, returns
Finance	payments, payment_transactions, refunds, invoices, invoice_items, tax_records, financial_adjustments
Media	video_categories, videos, video_assets, tags, vehicle mappings, watch_history
Support	support_tickets, ticket_messages, attachments, escalation_requests, escalation_history, assignments
System	notifications, preferences, logs, audit_logs, settings, idempotency_keys, background_jobs

Key data rules

- Use UUID primary keys and explicit foreign keys where appropriate.
- Maintain stock balances plus immutable stock movement records.
- Snapshot product name, SKU, price, tax and discount in order items.
- Keep order status and payment status separate.
- Do not delete completed service or financial history through ordinary UI actions.
- Scope VIN/registration uniqueness only after ownership and tenant rules are confirmed.
- Store file metadata in DB and file bytes in object storage.
- Use reviewed, versioned migrations.

9. Backend API Structure

Recommended base path: /api/v1. Use JSON, server-side validation, authenticated access, consistent errors, pagination and audit context.

Module	Example endpoints
Authentication	POST /auth/register • /auth/login • /auth/refresh • /auth/logout • GET /auth/me
Organizations	/organizations • /members • /roles • /permissions
Customers & vehicles	/customers/me • /vehicles • /vehicles/{id} • /vehicles/{id}/service-history
Service network	/service-centers • /service-centers/{id}/services
Bookings & work orders	/appointments • /appointments/{id}/status • /work-orders • /work-orders/{id}/tasks • /assignments
Parts & inventory	/parts • /categories • /compatibility • /inventory/stock • /inventory/movements • /inventory/transfers
Commerce	/carts • /orders • /orders/{id} • /orders/{id}/tracking
Payments	/payments/intents • /payments/webhooks/{provider} • /invoices
Media & support	/videos • /video-categories • /support/tickets • /escalations
Operations	/notifications • /admin/* • /reports/*

This endpoint catalogue is indicative, not a frozen API contract. Final schemas, validation, permissions and OpenAPI definitions must be completed before implementation.

10. Core Business Workflows

Customer registration & vehicle onboarding

- Create account and verify phone/email.
- Create profile; add vehicle or enter scanner flow.
- Validate identifiers and check duplicates.
- Confirm ownership under the approved rule; display vehicle in garage.

Service booking

- Select vehicle → service → centre → available slot → submit.
- Centre confirms, rejects or proposes reschedule.
- Notify customer; check-in creates or links a work order.

Work order & service completion

- Check in and inspect vehicle.
- Create work order/tasks; assign technician.
- Record diagnosis, notes and required parts.
- Reserve/issue parts with transactional stock movements.
- Perform service and quality check; generate report.
- Invoice/payment; deliver vehicle and update service history.

Inventory

- Stock-in from purchase/receipt; stock-out for sale or work order.
- Transfers record source and destination.
- Returns, damaged stock and adjustments require reason and permission.
- Every movement is traceable; prevent negative stock unless approved.

Online parts purchase

- Browse/search → compatibility → detail → cart → checkout.
- Server rechecks price and stock at order placement.
- Create order/payment intent; verify payment through provider callback.
- Update order and payment separately; notify and provide tracking.

Offline purchase

- Centre searches part and checks local stock.
- Create sale, accept approved payment method and issue receipt.
- Exact tax, returns and customer-identification rules are Pending.

Support & escalation

- Customer opens ticket with issue and optional attachments.
- Assign to support/centre staff and record work.
- Escalate to configured higher department if unresolved.
- Keep assignment and status history through resolution.

Video & learning

- Admin uploads media and metadata; validate and review.
- Map to vehicle models/service categories where appropriate.
- Publish with role-based access; optionally record watch history.

11. Security, Privacy & Compliance

- Secure authentication, password hashing or managed identity, account recovery and session expiry.
- RBAC plus organization and service-centre scoping; object-level authorization on every record.

- Super Admin MFA strongly recommended.
- HTTPS; managed secret storage; rate limiting; input validation; OWASP-aligned controls.
- Prevent Broken Object Level Authorization (BOLA); test cross-tenant access.
- Do not store card details; use hosted checkout/provider SDK and signed webhooks.
- Validate uploaded file type, size, MIME and content; use private storage and signed URLs.
- Never log passwords, OTPs, tokens, payment secrets or unnecessary personal data.
- Audit role changes, stock adjustments, payment verification, refunds and ownership changes.
- Provide privacy notice, consent records where needed, retention policy and user data request process.
- Obtain India-specific legal/privacy/tax review, including applicable DPDP requirements, before public launch.

Security rule: frontend role checks are for UX only. Backend and database access policy enforce the security boundary.

12. Integrations & External Dependencies

Integration	Purpose	Status
Payment gateway	Online payments, refunds, signed callbacks	Provider and methods Pending
SMS / OTP	Phone verification and alerts	Provider Pending
Email	Receipts and operational notifications	Provider Pending
Push	Android notifications	Provider/config Pending
Maps	Centre discovery and directions	Provider Pending
Storage / CDN	Images, documents and videos	Provider Pending
QR / barcode	Vehicle or asset identification	Scan target Pending
Telematics	Battery, GPS or live monitoring	Not assumed; Pending
WhatsApp	Customer contact or automated alerts	Workflow/provider Pending

13. QA, Testing Strategy & Release Quality

No testing has been executed as part of this planning document. This is the proposed QA strategy for the development phase.

Test layer	What it verifies	Suggested tools (not final)
Unit	Business rules, calculations, permissions and status transitions	Jest / Vitest
Component	UI components, forms, validation and states	React Testing Library
API / integration	Endpoint contracts, auth, database and integrations	Supertest / API runner
End-to-end	Customer, service-centre and admin journeys	Playwright; Maestro/Detox/Appium
Acceptance	Business owner confirms approved workflows	UAT scripts and sign-off
Security	RBAC, tenant isolation, BOLA, uploads, webhooks	SAST/DAST + manual review
Performance	Response times, concurrency, stress and soak	k6 or equivalent
Accessibility	Keyboard, contrast, labels, screen-reader	Automated + manual review

Critical test scenarios

- A customer cannot read or modify another customer's vehicle, booking or order.
- One organization cannot access another organization's data.
- Technician cannot change payment settings or grant admin permissions.
- Duplicate submission cannot create duplicate orders or charges.
- Payment changes only after trusted server-side verification.
- Concurrent stock deductions cannot oversell or create negative stock.

- Cancelled bookings release slots/parts according to policy.
- Weak network does not lose confirmed transactions.
- Unauthorized users cannot access private files or videos.
- Sensitive admin, stock and payment actions create audit records.

Defect severity

Severity	Definition	Release response
Critical / P0	Data breach, cross-tenant access, payment compromise, data loss, core outage	Block release; fix and retest.
High / P1	Major workflow broken; no safe workaround	Normally block release.
Medium / P2	Feature degraded with workaround	Fix or formally accept.
Low / P3	Minor visual/copy issue	Schedule if approved.

Definition of Done

- Approved requirement and acceptance criteria exist.
- Code review completed; automated checks pass.
- Relevant unit, API, integration and E2E tests pass.
- Security and tenant-isolation checks pass for affected areas.
- Responsive/accessibility checks completed.
- Logs, monitoring and failure handling are in place.
- Documentation and release notes updated.
- Product owner accepts the feature in staging.

14. Environments, Deployment & Operations

- Environments: Development → Staging/UAT → Production; isolate credentials and data.
- CI/CD should run lint, type checks, tests, security checks and controlled deployment.
- Use versioned DB migrations, backups and rollback/forward-fix procedures.
- Keep secrets out of source control and client bundles.
- Monitor API errors/latency, DB health, job failures, payment webhooks and storage.
- Use structured logs with request IDs; protect audit logs.
- Schedule encrypted backups and perform restore drills.
- Define incident response, support ownership and maintenance windows.

Operational decision	Status
Hosting provider / cloud region	Pending
Production domain and app-store accounts	Pending
Backup retention	Pending
RPO / RTO targets	Pending
Support hours and incident owner	Pending
Expected users / traffic	Pending
Budget and monthly operating ceiling	Pending

15. Delivery Roadmap & Project Phases

Phase	Deliverable	Planning status
1	Initial product/reference analysis	Draft completed
2	PRD & product architecture	Draft v0.1
3	UI/UX & screen architecture	Draft v0.1
4	Database logical architecture & ERD	Draft v0.1; physical schema Pending
5	Backend architecture & API catalogue	Draft v0.1
6	System integration & workflow architecture	Draft v0.1
7	Security, privacy & compliance architecture	Draft v0.1
8	Functional requirements & business rules	Draft v0.1
9	UI/UX design system & screen specifications	Draft v0.1
10	QA & quality assurance strategy	Outlined; detailed test cases Pending
11	Deployment, DevOps, hosting & release plan	Not completed
12	Estimation, milestones, resourcing & delivery plan	Not completed

After approval, a sensible build sequence is: foundation/authentication and organization model → customer/vehicle → service-centre and booking → work orders → parts/inventory → online/offline commerce → payments/integrations → media/support → reporting and hardening. Final order depends on the approved MVP.

16. Pending Client Decisions & Assumptions

These are business decisions that cannot safely be finalized by developers alone. Collect them in one structured client decision session; do not repeatedly ask questions already answered.

Topic	Required confirmation
Business model	Single business vs multi-organization/multi-brand operation.
EV scope	Supported vehicle types, brands, models and variants.
Scanner	What is scanned (vehicle QR, VIN, chassis, product barcode) and what action follows.
Monitoring	Service status, battery/health data, GPS/live tracking or other; device/API source required.
Video Assistant	Static videos, guided troubleshooting, live video or AI assistant.
Software Section Service	Diagnostics, configuration, firmware/OTA or another service.
Higher Department	Hierarchy, assignment rules, SLA and permissions.
Online payments	Gateway, UPI/cards/netbanking/COD, refunds and settlement.
Offline sales	Walk-in workflow, invoice/tax, customer details, returns and payment.
Inventory ownership	Central warehouse vs centre-owned stock; transfer/replenishment policy.
Service rules	Catalogue, pricing, labour charges, cancellation, warranty and appointment slots.
Support	Channels, hours, ownership and response expectations.
Legal & tax	Business entity, applicable GST/tax, privacy, retention and legal review.
Scale & budget	Expected users, centres, transactions, launch region, timeline and hosting budget.

Working assumptions — not approvals

- Multi-tenant-ready architecture is recommended; commercial model is Pending.
- Android + responsive website + admin and service-centre portals are proposed.
- PostgreSQL and Modular Monolith are recommended for the first release.
- Online and offline parts paths are proposed; operating rules remain Pending.
- No live telemetry, OTA control or AI diagnosis without explicit integration and approval.
- Payment, maps, OTP, messaging and storage providers are not selected.

17. Client & Developer Responsibilities

Client / Product Owner	Development Team
Confirm business rules, launch priorities and MVP scope.	Convert approved requirements into tasks and estimates.
Provide brand assets, legal content, product/service data and authorized media.	Design architecture, UX, APIs, database and implementation plan.
Select/approve payment, messaging, maps and other providers.	Implement secure integrations and document dependencies.
Review prototypes and staging workflows; provide acceptance/sign-off.	Provide test evidence, defect reports, release notes and runbooks.
Approve budget, timeline, changes and production launch.	Protect data; implement access controls, backups, monitoring and incident handling.

Neither party should treat a draft proposal as approved scope. Record decisions and approvals with owner, date and version.

18. Sign-off & Change Control

- Product scope and MVP approved.
- User roles and permissions approved.
- Business workflows and status definitions approved.
- UI/UX prototype approved.
- Database and API specifications approved.
- Security/privacy/legal review completed.
- QA acceptance criteria and UAT plan approved.
- Budget, timeline, hosting and launch plan approved.

Every change after sign-off should record: description, reason, affected modules, data/API impact, security impact, cost/timeline impact, decision owner and approval date.

END OF BLUEPRINT — Draft v0.1